



UNIVERSITAS
GADJAH MADA

Priority Based Schedule Conflict Resolver and Free Slot Recommender Using C++

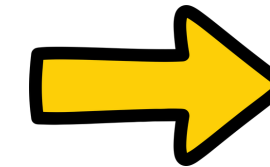
A Smart Solution for Managing and Resolving
Schedule Conflicts

Group 4 : Rifat Adly Pratama, Zia Maharani Putri A. , Valerie Graceline Darsojo

Problem



Complex and overlapping commitments, including lectures, deadlines, and extracurriculars



Limitation of existing tools



"Time Poverty" or Insufficient time in managing college work



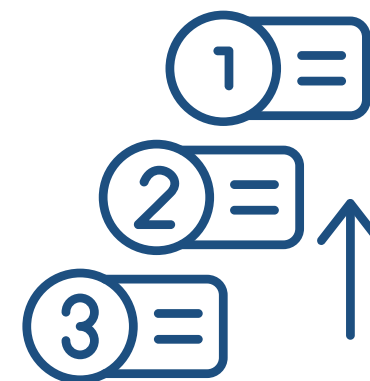
Note: this program does not aim to replace existing applications nor solve time poverty directly, but to help the user make more structured scheduling decisions

Solution

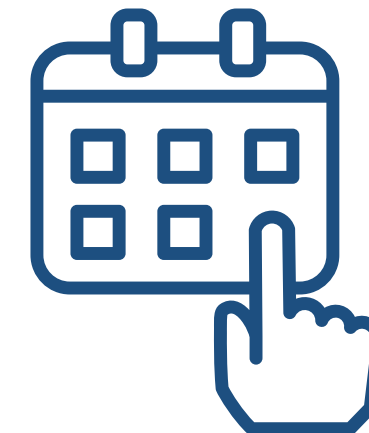
A console based C++ schedule manager for **student**. The system stores each activity title, start time, end time, priority level from 1 to 5, and activity type.



Automated **time**
collision detection



Priority-driven
conflict resolution



Recommends
free slot for
flexible activities

Scope of Our Project

Aspect	Project Scope
Target User	Students and other individuals who need to manage overlapping activities.
Schedule range	The system can store and manage activities across a 7 day weekly schedule (e.g., Monday to Sunday).
Time format	24 hour format
Priority	1 = lowest priority, 5 = highest priority
Fixed activity	Cannot be moved by the system
Flexible activity	Can receive alternative slot recommendations
Conflict handling	The system recommends options; it does not reschedule automatically
Output	Final schedule, detected conflicts, and recommended free slots
Storage	Text file using <u><code>fstream</code></u>
Conflict Tolerance	Overlaps of ≤ 10 minutes trigger a warning with a user option to ignore or relocate.

Algorithmic Approach

System Input Parameters

- Activity title
- Day of the week (1 to 7)
- Start time & End time
- Priority level from 1 to 5
- Activity type (fixed or flexible)

Core Algorithm

The system uses a priority based
¹greedy heuristic.

1. an algorithmic strategy that makes locally optimal decisions at each step by ranking and selecting items based on assigned urgency or value

Algorithmic Approach

Conflict Resolution Rules

1. **Fixed vs. Flexible:** The system keeps the fixed activity.
2. **Flexible vs. Flexible:** The system keeps the higher priority activity.
3. **Equal Priority:** The system pauses automatic choice and asks the user.
4. **Minor Overlap Handling:** If the collision duration is ≤ 10 m, the system asks the user to either “Ignore” or “Resolve” the conflict before applying further rules.
5. **Displacement & Relocation:** The lower priority activity is flagged for rescheduling. The system searches for free slots matching the displaced activity’s duration.

Key Features

Free Slot Recommendation

A free slot is an empty time interval that satisfies the following parameters:

- **Does not overlap** with existing scheduled activities.
- Has a **duration** equal to or longer than the displaced activity.
- Falls within the predefined schedule boundary (e.g., 07:00 - 22:00).
- Can be interactively **selected** by the user as an alternative schedule.

Exp:

Day	Existing Schedule	Activity to Move	Valid Recommendation
Monday	Lunch/Activity: 12:00 - 15:00	Group Meeting: 90 minutes	Monday, 15:00 - 16:30
Tuesday	Free time: 18:00 - 19:00	Group Meeting: 90 minutes	Not valid, duration is too short

C++ Implementation Components

struct / class

Std::vector

Std::sort

Std::fstream

Key Features

Collision Detection Logic

Two activities overlap when:

$$(Start_A < End_B) \wedge (End_A > Start_B)$$

Example:

Day	Activity	Time	Result
Monday	Algorithms Class	08:00 - 10:00	-
Monday	Group Meeting	09:00 - 10:30	Conflict detected
Tuesday	Student Club Activity	08:00 - 10:00	No conflict

Key Features

Conflict Tolerance

Two activities get tolerance when:

$$\text{Overlap_Duration} = \min(\text{End}_A, \text{End}_B) - \max(\text{Start}_A, \text{Start}_B) \leq 10 \text{ minutes}$$

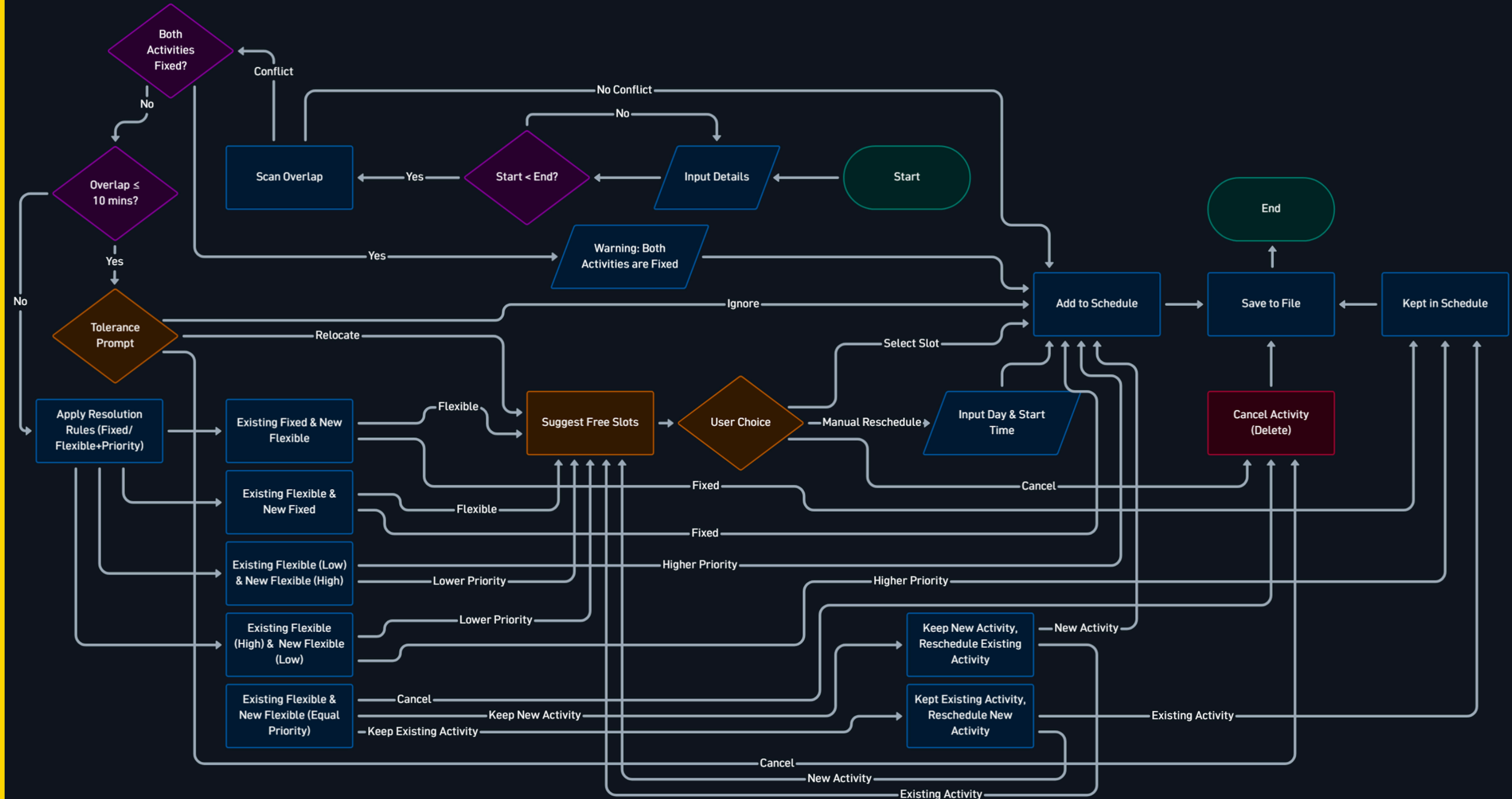
Example:

Day	Activity	Time	Result
Monday	Algorithms Class	07:00 - 8:00	-
Monday	Committee Briefing	07:55 - 09:00	Minor Conflict (5 mins) → System triggers a prompt: "Ignore or Relocate?"

System Workflow

1. **Input**: Activity details (activity name, day of the week, start/end times, priority scale 1-5, activity type).
2. **Validation**: Start time must be earlier than end time
3. **Scan**: Adjacent items evaluated using the overlap logic condition.
4. **Resolve & Relocate**:
 - a. If there is **no conflict**, the activity is added to the schedule
 - b. If a **minor conflict occurs** (≤ 10 mins), the system triggers the Conflict Tolerance prompt, allowing the user to either “Ignore” or “Relocate”.
 - c. If a **major conflict occurs** (> 10 mins), the system automatically applies the conflict resolution rules based on priority and activity type.
 - d. If an **activity is displaced**, the user may select an alternative slot from the system’s recommendations, or cancel the activity.
5. **Save**: Writes the finalized schedule to a local file storage.

Flowchart





UNIVERSITAS
GADJAH MADA

Thank You For Your Attention!

References :

1. Wladis, C., Hachey, A. C., & Conway, K. M. (2024). *It's about time, part II: Does time poverty contribute to inequitable college outcomes by gender and race/ethnicity?*. *AERA Open*, 10, 23328584241237971.
2. Abdipoor, S., Yaakob, R., Goh, S. L., & Abdullah, S. (2023). *Meta-heuristic approaches for the University Course Timetabling Problem*. *Intelligent Systems with Applications*, 19, Article 200253. <https://doi.org/10.1016/j.iswa.2023.200253>
3. Babaei, H., Karimpour, J., & Hadidi, A. (2015). *A survey of approaches for university course timetabling problem*. *Computers & Industrial Engineering*, 86, 43–59. <https://doi.org/10.1016/j.cie.2014.11.010>